

Language Models for Algorithmic Problems. An algorithmic problem is a problem that may be solved by an algorithm. In this paper, we focus on algorithmic problems $f : V^+ \rightarrow V_A$ that asks for computing $f(\mathcal{S}_{\text{in}})$ given a sequence of tokens $\mathcal{S}_{\text{in}}$ as the input, where V_A is the set of possible answers. We say that an LM M can (directly) solve an algorithmic task f if, given the sequence $\mathcal{S}_{\text{in}}$, the probability distribution $M(\mathcal{S}_{\text{in}})$ for the next token is peaked at the correct output token $f(\mathcal{S}_{\text{in}})$, i.e., $\arg \max_{j \in V} M(\mathcal{S}_{\text{in}})[j] = f(\mathcal{S}_{\text{in}})$.

Chain-of-Thought Reasoning. *Chain-of-Thought* (CoT) reasoning allows the LM to produce intermediate steps before the final output. Following Feng et al. (2023); Li et al. (2024), our paper studies the effectiveness of CoT reasoning in improving the expressiveness of LMs, and we allow the intermediate steps to be an arbitrary sequence of tokens. We say that an LM M can solve an algorithmic problem f with CoT if the following process terminates with a sequence ended with $f(\mathcal{S}_{\text{in}})$. First, let $\mathcal{S}_0 = \mathcal{S}_{\text{in}}$. For all $i \geq 0$, decode the next token $s_i^{\text{next}} = \arg \max_{j \in V} M(\mathcal{S}_i)[j]$ from M , and append it to the current sequence $\mathcal{S}_{i+1} = \mathcal{S}_i \oplus s_i^{\text{next}}$. If $s_i^{\text{next}} \in V_A$, then the process terminates with $\mathcal{S}_{i+1}$ with i steps of CoT; otherwise the process continues.

It is evident that if an LM can solve an algorithm problem with 0 steps of CoT, then an LM M can (directly) solve the problem. In this case, we also say that the LM can solve the problem without CoT.

4 Can CoT improve the Representation Power of RNNs?

In this section, we aim to understand the representation power of RNNs with CoT. We first show the positive result that RNNs with CoT can solve tasks that are impossible for RNNs without CoT fixing the state size. We then proceed to understand whether CoT can make RNNs as expressive as Transformers. We show that, even with CoT, RNNs still struggle to solve problems that explicitly require in-context retrieval and this representation gap propagates to seemingly retrieval-irrelevant reasoning tasks such as IsTree. Finally, we show that this gap is indeed one-sided: there only exist tasks where Transformers require exponentially less parameters than RNNs, but not the other way around.

4.1 CoT Strictly Improves RNNs

On the positive side, we show that CoT broadens the representation power of RNNs under mild complexity assumption.

Theorem 4.1. *Assuming $\text{PSPACE} \not\subseteq \text{P/poly}$, there exists an algorithmic problem such that (1) there exist constant-size Linear RNNs that can solve the problem with polynomial length CoT; and (2) any constant-size regular RNNs cannot solve the problem without CoT.*

See Appendix C.5 for the proof. The key insight is that the representation power of RNNs without CoT is limited to shallow circuits of size $\text{poly}(\log n)$, but RNNs with CoT can simulate $O(\log n)$ -space Turing machines perfectly with $\text{poly}(n)$ steps. This result is consistent with previous works on the benefit of CoT for Transformers (Feng et al., 2023; Li et al., 2024), which also prove based on mild complexity assumptions that Transformers with CoT have the representation power to simulate polynomial-size circuits to solve all the problems in P but Transformers without CoT cannot. Here we rigorously prove that a similar benefit of CoT also applies to RNNs, but a key difference is that CoT cannot boost the representation power of RNNs to simulate every polynomial-size circuit family.

4.2 CoT Cannot Close the Representation Gap with Transformers

Now we proceed to understand whether CoT can make RNNs as expressive as Transformers. The answer turns out to be negative: RNNs, even with CoT, struggle to solve very simple algorithmic problems that require the capability of retrieving information from the current context, which we call *In-context Retrieval* for short. This limitation is directly related to the memory efficiency of RNNs. For a model with at most $o(n)$ bits in memory, we can involve techniques from streaming complexity to prove impossibility results for problems requiring In-context Retrieval.

4.2.1 Simple Problems on In-Context Retrieval

Here we list several simple algorithmic problems that directly test the in-context retrieval capability of the model, which turn out to be a good test-bed for understanding the limitations of RNNs compared to Transformers.

Definition 4.2 (Index). Index is a problem that given a sequence of tokens with length n and a query token $i \in [n]$, requires the model to output the type of the i -th token.

Definition 4.3 (Associative Recall). Associative Recall (AR) is a problem that given a sequence of tokens with length n consisting of tokens in $[n]$ and a query token $q \in [n]$, requires the model to output the next token of q in the sequence.

Definition 4.4 (*c*-gram Retrieval). An *c*-gram is a contiguous subsequence of *c* tokens in a sequence of tokens. *c*-gram retrieval is a problem that given a sequence of tokens with length *n* and a query (*c* − 1)-gram that is the prefix of a *c*-gram in the sequence, requires the model to output the last token of that *c*-gram.

Definition 4.5 (Counting). Counting is a problem that given a sequence of tokens with length *n*, a query token $q \in [n]$, and a query number $t \in \mathbb{N}$, requires the model to output 0 or 1 to indicate whether the number of occurrences of *q* in the sequence is greater than *t*.

Here, Index and AR are perhaps the most basic problems in retrieval, where Index asks for retrieving a token from the input sequence viewed as a linear array of tokens, and AR asks for retrieving a token from the input sequence viewed as an associative array. These two problems have been studied extensively by different communities. Index is a classic problem in streaming and communication complexity (Munro & Paterson, 1980), known to be impossible to solve with $o(n)$ bits of memory for streaming algorithms. AR has been regarded as a fundamental problem that an artificial intelligence system should be able to solve (Willshaw et al., 1969; Hopfield, 1982; Hinton & Anderson, 2014; Graves et al., 2014; Ba et al., 2016). In the context of LLMs, AR has been observed to correlate with in-context learning performance (Elhage et al., 2021) and has also been used extensively as synthetic surrogate tasks for pretraining performance (Fu et al., 2023; Poli et al., 2023; Lutati et al., 2023). Besides Index and AR, *c*-gram retrieval is a natural extension of AR to the case where the query key can contain multiple tokens: instead of retrieving a token given a single-token key, *c*-gram retrieval asks for retrieving a token when the given key is a (*c* − 1)-gram. This task has been studied empirically, but not theoretically in Jelassi et al. (2024). Counting is a problem that asks for the number of occurrences of a token, thereby testing the model’s capability to retrieve some statistics of relevant information from the input sequence.

The following theorems show that constant-size RNNs cannot solve any of the four tasks when the context length is long, while constant-size Transformers can solve them perfectly.

Theorem 4.6. *For task $T \in \{\text{Index, AR, } c\text{-gram retrieval, Counting}\}$, there exist constant-size Transformers that can solve T . On the other hand, any RNN with $o(n)$ -bit memory cannot solve T of size n with any length of CoT for large enough n .*

We note that Theorem 4.6 does **not** imply that RNNs are incapable of in-context retrieval at all. Instead, it states that the maximal context length that RNNs can effectively retrieve from is linear in its state size. Although for a short context window, RNNs can be trained to perform in-context retrieval (see e.g. Arora et al. (2023); Gu & Dao (2023)), this limitation in retrieval capabilities has been empirically observed: for example in Waleffe et al. (2024), both pretrained Mamba and Mamba-2 7B models are shown to have significantly worse Phonebook-retrieval capabilities on 1K context length than Transformers with the same size and trained on the same data.

Proof Sketch. The key idea of the lower bound of the proof is to put RNNs into the framework of communication complexity and use information-theoretic arguments to prove a lower bound. RNNs have the following property if party A simulates the RNN on the first part of the input and sends the state to party B, then party B can simulate the RNN on the second part of the input with the state received from party A to recover the output of the RNN perfectly. Hence, in the above two theorems, if the RNN can solve the problem with $o(n)$ input size, then the information about the input can be compressed to $o(n)$ bit to produce the output, which contradicts the information-theoretic lower bound.

For the upper bound, we show that the Transformer can solve the problem by utilizing an attention mechanism called Match that takes the query token and attends to previous keys that match the query token on certain predefined coordinates. This mechanism allows the Transformer to read its context window like a key-value dictionary and hence can solve the problems perfectly. For the counting problem, we additionally use a COUNT attention mechanism that counts the number of occurrences of the queried token by attending evenly to each appearance of the queried token. $\square$

4.2.2 Understanding the Representation Power of RNNs Beyond Simple In-context Retrieval Problems

A natural question would be if an algorithmic problem does not directly test the in-context retrieval capability, can we hope that RNNs would have the representation power to solve it? Do RNNs and Transformers have the same representation power in this case? We show that the limited memory size in RNNs can still be a bottleneck in solving algorithmic problems. Even if the retrieval capability is not explicitly tested in an algorithmic problem, it may still be required implicitly for reasoning about the answer.

We demonstrate this gap on a minimal example of algorithmic problems, called IsTree: given an undirected graph G of n nodes, determine whether G is a tree, i.e., whether every pair of nodes is connected by exactly one simple path. A classical solution to IsTree is running Depth First Search (DFS), which takes $O(n)$ time.

In the context of language modeling, we can write the graph G as a sequence of tokens, and then the task of IsTree is to determine whether G is a tree by predicting a YES/NO token with or without CoT. We use the following tokenization

for the graph G :

$$\text{Tokenize}(G) = \{ \langle s \rangle, u_1, \sim, v_1, u_2, \sim, v_2, \dots, u_m, \sim, v_m \}, \quad (4)$$

where $\langle s \rangle$ and $\sim$ are two special tokens representing the start of the sentence and an edge, and u_i, v_i are numbers denoting the nodes of the graph.

Our result states that RNN with $o(n)$ bit memory cannot solve IsTree, even with an arbitrary choice of chain of thought. On the other hand, there exists a Transformer with constant size and $O(\log n)$ precision that can generate a chain-of-thought of length n following DFS and perfectly solve the same question.

Theorem 4.7. *There exist constant-size Transformers that can solve IsTree with CoT of length $O(n)$. On the other hand, any RNN with $o(n)$ -bit memory cannot solve IsTree with any length of CoT.*

Proof Sketch. The main idea of the proof is that the task of IsTree requires the model to reason about the global structure of the graph, which is beyond the capability of RNNs with limited memory. We prove the lower bound by constructing a graph from a binary sequence and showing that RNNs with $o(n)$ memory cannot solve the problem by a reduction to an information-theoretic lower bound. For the upper bound, we show that the Transformer can simulate the DFS algorithm by outputting the Euler tour of the connected components of vertex 1 and then check the length of the Euler tour with its capability of In-context Retrieval.

The key idea of the lower bound of the proof is to again utilize the information-theoretic lower bound. This idea lies in the core of streaming complexity literature and investigation on the IsTree problem dates back to [Henzinger et al. \(1998\)](#). We hereby restate the proof for completeness. Given any binary sequence x of length $n - 2$ and an index $k \in [n - 3]$, we can construct a graph as follows: the graph has n nodes, and vertex a is connected to vertex $x_a + n - 1$ for any $a \in [n - 2]$. Moreover, vertex k is connected to vertex $k + 1$. The graph is a tree if and only if $x_k \neq x_{k+1}$. Now assuming there is an RNN with $o(n)$ memory that can solve IsTree, consider two parties A and B each holding

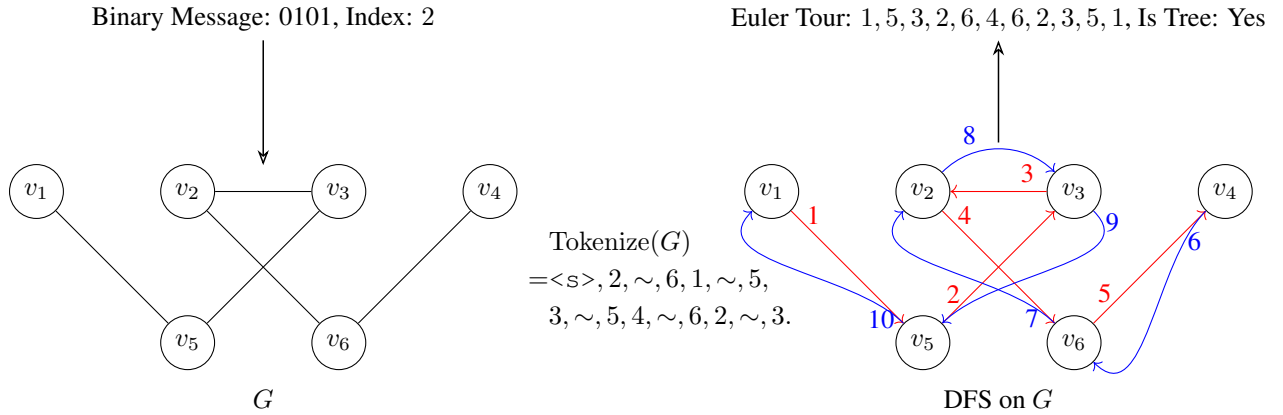


Figure 3: An example of the graph constructed from the binary sequence $x = 0101$ and the index $k = 2$ and the corresponding DFS tour.

the sequence x and the index k , they can construct two parts of the graph using their information and then A can simulate RNN on the first part of the graph and send the state to B, and B can simulate RNN (potentially with CoT) on the second part of the graph to recover the output of the IsTree problem, which is equivalent to whether $x_k \neq x_{k+1}$. However, note that k is never sent to A, and hence actually B can get whether $x_k \neq x_{k+1}$ for any $k \in [n - 3]$, which contradicts the information-theoretic lower bound.

Now for the upper bound, we will let the Transformer simulate the DFS algorithm by outputting the Euler tour of the connected components of vertex 1 and then check the length of the Euler tour (see Algorithm 1). To simulate the tour, we will implement two functions through the Transformer Block:

1. Given a prefix of the tour, find the parent of the last vertex in the tour. This can be implemented by copying each token's predecessor's type to that token and then using the Match mechanism to match the first occurrence of the current token in the sequence.
2. Given the tokenized input graph and an edge (u, v) , find the next edge after (u, v) containing v in the input graph. We will use another attention mechanism called COUNT to count, for each edge $e = (a, b)$ in tokenized input